Numerical Methods & Applied Programming (ENG2124)

Pandas for Datasets

Dr Ian (Yinglong) He

A: 14b AC 03

E: yh0015@surrey.ac.uk

T: 01483 688784

Content

- What is Pandas?
- Pandas data structures: Series & DataFrame
- Operations
- Data indexing and selection
- Handling missing data
- Merging data
- Grouping data
- File input/output

What is Pandas?

Pandas ("Python Data Analysis")

- Pandas can deal with ***datasets***, including tabular, sequential, homogeneous, and heterogeneous data.
- Pandas is built on top of the NumPy and created by Wes McKinney in 2008
- It has functions for analysing, cleaning, exploring, and manipulating data.
- It has two primary data structures: `Series` (1-dimensional) and `DataFrame` (2-dimensional).
- Source code: https://github.com/pandas-dev/pandas.

Pandas data structures

- Pandas objects can be thought of as enhanced versions of NumPy structured arrays, in which the rows and columns are identified with labels rather than simple integer indices.
- A Pandas `Series` is an analog of a one-dimensional array with *flexible indices*.
- A Pandas `DataFrame` is an analog of a two-dimensional array with both *flexible row indices* and *flexible column names*.

Pandas data structures

Pandas Series

- Start with the standard NumPy and Pandas imports

```
import numpy as np
import pandas as pd
```

- Created Series from a list or array

```
s = pd.Series([0.25, 0.5, 0.75, 1.0])
print(s)
```

```
0    0.25
1    0.50
2    0.75
3    1.00
dtype: float64
```

Pandas data structures

- Explicit index

```
s = pd.Series([0.25, 0.5, 0.75, 1.0],
              index=['a', 'b', 'c', 'd'])
print(s)
```

```
a    0.25
b    0.50
c    0.75
d    1.00
dtype: float64
```

- Series as a special dictionary

```
# A dictionary (population by state)
pop_dict = {'California': 38332521,
            'Texas': 26448193,
            'New York': 19651127,
            'Florida': 19552860,
            'Illinois': 12882135}
# Create a Series
pop_s = pd.Series(pop_dict)
print(pop_s)
```

```
California    38332521
Texas         26448193
New York      19651127
Florida       19552860
Illinois      12882135
dtype: int64
```

Pandas data structures

- Access and slice Series data

```
pop_s['California']
```

```
38332521
```

```
pop_s['California':'New York']
```

```
California    38332521
Texas         26448193
New York      19651127
dtype: int64
```

Slicing with explicit labels: last included

VS

Slicing with integer indices: last not included

Pandas data structures

Pandas DataFrame

- Created DataFrame from a dictionary of list

```
d = {'col1': [1, 2], 'col2': [3, 4]}
df = pd.DataFrame(d)
print(df)
```

```
    col1    col2
0   1       3
1   2       4
```

- Created DataFrame from a dictionary of Series

```
author = pd.Series(['Jitender', 'Purnima', 'Arpit', 'Jyoti'])
article = pd.Series([210, 211, 114, 178])
d = {'author': author, 'article': article}
df = pd.DataFrame(d)
print(df)
```

```
     author  article
0  Jitender      210
1   Purnima      211
2     Arpit      114
3     Jyoti      178
```

Pandas data structures

- Created DataFrame from a NumPy ndarray

```
arr = np.array(
    [[-0.730396, 1.444362, 0.026828, 0.155011],
    [0.595145, -0.516760, 0.617344, 0.139345],
    [-1.151051, 1.341014, -0.530525, -0.382772],
    [0.973117, 0.233997, -1.794288, 0.097118],
    [-0.292818, -0.196199, 0.347357, 1.529342],
    [0.252646, -0.088004, -1.275728, 0.365296]])

df = pd.DataFrame(arr, columns=['a', 'b', 'c', 'd'])
print(df)
```

```
    a          b          c          d
0   -0.730396  1.444362   0.026828   0.155011
1   0.595145   -0.516760  0.617344   0.139345
2   -1.151051  1.341014   -0.530525  -0.382772
3   0.973117   0.233997   -1.794288  0.097118
4   -0.292818  -0.196199  0.347357   1.529342
5   0.252646   -0.088004  -1.275728  0.365296
```

Pandas data structures

- Display the first or last *N* (default 5) rows of DataFrame

```
df.head()
```

```
    a           b           c           d
0   -0.730396   1.444362    0.026828    0.155011
1   0.595145    -0.516760   0.617344    0.139345
2   -1.151051   1.341014    -0.530525   -0.382772
3   0.973117    0.233997    -1.794288   0.097118
4   -0.292818   -0.196199   0.347357    1.529342
```

```
df.tail(3)
```

```
    a           b           c           d
3   0.973117    0.233997    -1.794288   0.097118
4   -0.292818   -0.196199   0.347357    1.529342
5   0.252646    -0.088004   -1.275728   0.365296
```

Pandas data structures

- Display the index, columns, and data of DataFrame

```
df.index
```

```
RangeIndex(start=0, stop=6, step=1)
```

```
df.columns
```

```
Index(['a', 'b', 'c', 'd'], dtype='object')
```

```
df.values
```

```
array([[-0.730396,  1.444362,  0.026828,  0.155011],
       [ 0.595145, -0.51676 ,  0.617344,  0.139345],
       [-1.151051,  1.341014, -0.530525, -0.382772],
       [ 0.973117,  0.233997, -1.794288,  0.097118],
       [-0.292818, -0.196199,  0.347357,  1.529342],
       [ 0.252646, -0.088004, -1.275728,  0.365296]])
```

Pandas data structures

- Generate descriptive statistics of DataFrame.

```
df.describe()
```

```
        a          b          c          d
count   6.000000   6.000000   6.000000   6.000000
mean    -0.058893  0.369735   -0.434835  0.317223
std     0.810964   0.828530   0.948304   0.643107
min     -1.151051  -0.516760  -1.794288  -0.382772
25%     -0.621002  -0.169150  -1.089427  0.107675
50%     -0.020086  0.072996   -0.251848  0.147178
75%     0.509520   1.064260   0.267225   0.312725
max     0.973117   1.444362   0.617344   1.529342
```

Pandas data structures

- Sort DataFrame by the index

```
df.sort_index(axis=0, ascending=False)
```

```
    a           b           c           d
5   0.252646    -0.088004   -1.275728   0.365296
4   -0.292818   -0.196199   0.347357    1.529342
3   0.973117    0.233997    -1.794288   0.097118
2   -1.151051   1.341014    -0.530525   -0.382772
1   0.595145    -0.516760   0.617344    0.139345
0   -0.730396   1.444362    0.026828    0.155011
```

- Sort DataFrame by a column

```
df.sort_values(by='b')
```

```
    a           b           c           d
1   0.595145    -0.516760   0.617344    0.139345
4   -0.292818   -0.196199   0.347357    1.529342
5   0.252646    -0.088004   -1.275728   0.365296
3   0.973117    0.233997    -1.794288   0.097118
2   -1.151051   1.341014    -0.530525   -0.382772
0   -0.730396   1.444362    0.026828    0.155011
```

Operations

Descriptive statistics of DataFrame

- Across axis 0 (rows), i.e., column mean

```
df.mean()
```

```
a   -0.058893
b    0.369735
c   -0.434835
d    0.317223
dtype: float64
```

- Across axis 1 (column), i.e., row mean

```
df.mean(1)
```

```
0    0.223951
1    0.208769
2   -0.180834
3   -0.122514
4    0.346920
5   -0.186447
dtype: float64
```

Operations

apply method

- Apply a function along an axis (default 0, apply function to each column) of the DataFrame.

```
df.apply(np.cumsum)
```

```
    a           b           c           d
0   -0.730396   1.444362    0.026828    0.155011
1   -0.135252   0.927601    0.644172    0.294356
2   -1.286303   2.268616    0.113647    -0.088417
3   -0.313186   2.502613    -1.680641   0.008702
4   -0.606003   2.306414    -1.333284   1.538044
5   -0.353358   2.218409    -2.609012   1.903340
```

Data indexing and selection

Indexers: loc and iloc

- loc allows indexing and slicing that always references the explicit (*label based*) index.
- iloc allows indexing and slicing that always references the implicit (*position based*) index.

```
df.loc[:3, ['a', 'c']]
```

	a	c
0	-0.730396	0.026828
1	0.595145	0.617344
2	-1.151051	-0.530525
3	0.973117	-1.794288

Slicing with explicit labels: last included

```
df.iloc[:3, 0:3]
```

	a	b	c
0	-0.730396	1.444362	0.026828
1	0.595145	-0.516760	0.617344
2	-1.151051	1.341014	-0.530525

Data indexing and selection

Boolean indexing

```
df['b'] >= 0
```

```
0     True
1    False
2     True
3     True
4    False
5    False
Name: b, dtype: bool
```

```
df.loc[df['b'] >= 0]
```

	a	b	c	d
0	-0.730396	1.444362	0.026828	0.155011
2	-1.151051	1.341014	-0.530525	-0.382772
3	0.973117	0.233997	-1.794288	0.097118

Handling missing data

- There are several useful methods for detecting, removing, and replacing null values in Pandas data structures:

  To detect

  - isnull: Generate a boolean mask indicating missing values
  - notnull: Opposite of isnull

  To remove

  - dropna: Return a filtered version of the data

  To replace

  - fillna: Return a copy of the data with missing values filled or imputed

Handling missing data

```
df = pd.DataFrame([[1,      np.nan, 2],
                   [2,      3,      5],
                   [np.nan, 4,      6]])
print(df)
```

```
    0   1   2
0   1.0 NaN 2
1   2.0 3.0 5
2   NaN 4.0 6
```

NaN – Not a Number

- By default, `dropna` drops all rows including any null value:

```
df.dropna()
```

```
    0   1   2
1   2.0 3.0 5
```

- We can also drop all columns containing a null value:

```
df.dropna(axis='columns')
```

```
    2
0   2
1   5
2   6
```

Handling missing data

- fillna fills null values with a single value, such as zero:

```
df.fillna(0)
```

```
    0   1   2
0   1.0 0.0 2
1   2.0 3.0 5
2   0.0 4.0 6
```

- A forward-fill to propagate the previous value forward

```
df.fillna(method='ffill')
```

```
    0   1   2
0   1.0 NaN 2
1   2.0 3.0 5
2   2.0 4.0 6
```

- A back-fill to propagate the next values backward

```
df.fillna(method='bfill')
```

```
    0   1   2
0   1.0 3.0 2
1   2.0 3.0 5
2   NaN 4.0 6
```

Merging data

- Some of the most interesting studies of data come from combining different data sources.

merge method

```
df_left = pd.DataFrame({
    'name': ['tom', 'mike'],
    'age': [20, 25]})
print(df_left)
```

```
    name    age
0   tom     20
1   mike    25
```

```
df_right = pd.DataFrame({
    'name': ['tom', 'mike'],
    'score': [68, 75]})
print(df_right)
```

```
    name    score
0   tom     68
1   mike    75
```

```
pd.merge(df_left, df_right, on='name')
```

```
    name    age score
0   tom     20  68
1   mike    25  75
```

Merging data

concat method

```
df1 = pd.DataFrame(np.random.randn(3, 2),
            columns=['A', 'B'])
print(df1)
```

```
    A           B
0   -0.553574   0.043440
1   -0.306701   0.986610
2   -0.499350   0.760573
```

```
df2 = pd.DataFrame(np.random.randn(3, 2),
            columns=['A', 'B'])
print(df2)
```

```
    A           B
0   -1.855219   0.730421
1   0.365833    -0.065586
2   1.012344    1.533554
```

```
pd.concat([df1, df2])
```

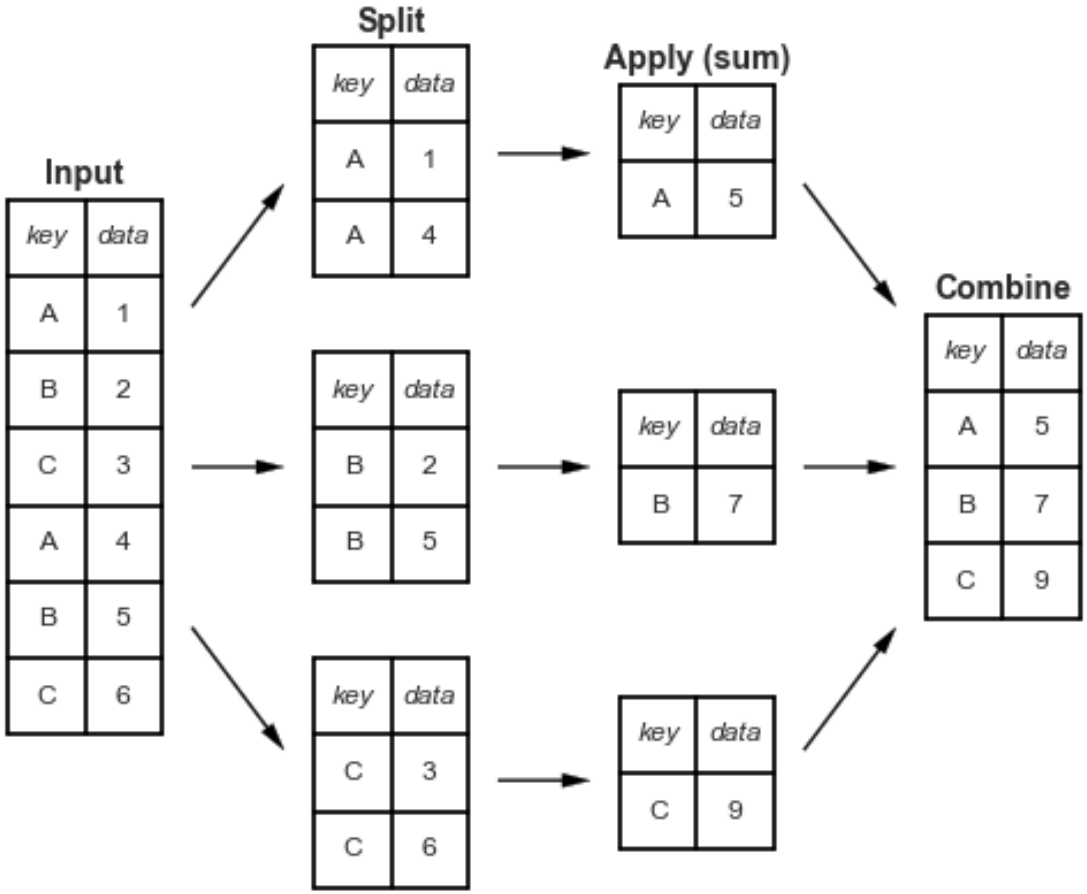
```
    A           B
0   -0.553574   0.043440
1   -0.306701   0.986610
2   -0.499350   0.760573
0   -1.855219   0.730421
1   0.365833    -0.065586
2   1.012344    1.533554
```

Grouping data

groupby method

- A groupby operation involves some combination of splitting the object, applying a function, and combining the results.
  - The *split* step involves breaking up and grouping a DataFrame depending on the value of the specified key.
  - The *apply* step involves computing some function, usually an aggregate, transformation, or filtering, within the individual groups.
  - The *combine* step merges the results of these operations into an output array.

Grouping data



Grouping data

Example: groupby

```
df = pd.DataFrame({
    'A': ['boy', 'girl', 'boy', 'girl',
          'girl', 'boy', 'girl', 'boy'],
    'B': ['one', 'two', 'one', 'three',
          'two', 'three', 'one', 'two'],
    'C': np.random.randn(8),
    'D': np.random.randn(8)})
print(df)
```

```
    A       B       C           D
0   boy     one     -0.484312   1.635437
1   girl    two     -0.575688   0.425640
2   boy     one     -0.234775   0.423848
3   girl    three   -0.167580   -1.603972
4   girl    two     0.405594    -1.260229
5   boy     three   0.709898    0.046353
6   girl    one     -1.479200   -0.542602
7   boy     two     0.963931    -0.098694
```

```
df.groupby(['A', 'B']).sum()
```

```
                C           D
A       B
boy     one     -0.719087   2.059285
        three   0.709898    0.046353
        two     0.963931    -0.098694
girl    one     -1.479200   -0.542602
        three   -0.167580   -1.603972
        two     -0.170094   -0.834588
```

File input/output

- One crucial feature of Pandas is its ability to write and read Excel, CSV, and many other types of files.

```
# Save DataFrame as a CSV file
df.to_csv('data.csv’)
```

```
# Load data from a CSV file
df = pd.read_csv('data.csv')
```

```
# Save DataFrame in an Excel file
df.to_excel('data.xlsx’)
```

```
# Load data from an Excel file
df = pd.read_excel('data.xlsx')
```